

Import Libraries

```
import tensorflow as tf
from tensorflow.keras.layers import (
    Input,
    Dense,
    Embedding,
    Flatten,
    Concatenate,
    Add
)
from tensorflow.keras.models import Model
```

Task 1: Movie Recommendation Neural Network

Problem

Inputs:

- User Age
- Favorite Genre (Integer)

Output:

- Predicted Movie Rating (1–5)

Code

```
import tensorflow as tf
from tensorflow.keras.layers import Input, Dense, Embedding, Flatten, Concatenate
from tensorflow.keras.models import Model

# Input 1: User Age
age_input = Input(shape=(1,), name="Age")

# Input 2: Favorite Genre (Integer)
genre_input = Input(shape=(1,), name="Genre")

# Genre Embedding
genre_embedding = Embedding(
    input_dim=10,
    output_dim=4
)(genre_input)

genre_embedding = Flatten()(genre_embedding)
```

```

# Combine Inputs
combined = Concatenate()([
    age_input,
    genre_embedding
])

# Hidden Layers
x = Dense(32, activation="relu")(combined)
x = Dense(16, activation="relu")(x)

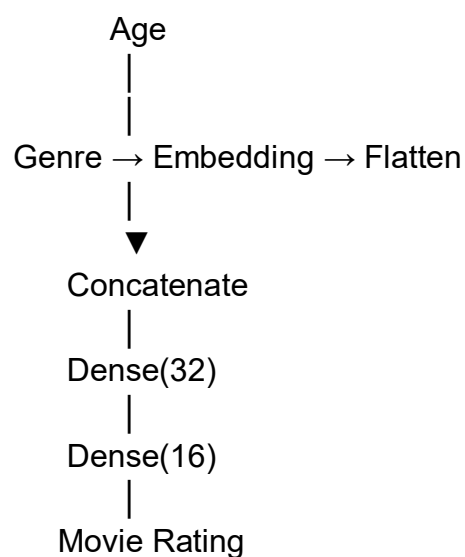
# Output Layer
rating_output = Dense(
    1,
    activation="linear",
    name="Movie_Rating"
)(x)

# Create Model
model = Model(
    inputs=[age_input, genre_input],
    outputs=rating_output
)

model.summary()

```

Architecture



Task 2: Multi-Output Network (Zomato)

Inputs

- Order Amount
- Distance

Outputs

- Delivery Time
- Delivery Rating

```
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
```

```
amount = Input(shape=(1,), name="Order_Amount")
```

```
distance = Input(shape=(1,), name="Distance")
```

```
merged = Concatenate()([
    amount,
    distance
])
```

```
hidden = Dense(
    32,
    activation="relu"
)(merged)
```

```
hidden = Dense(
    16,
    activation="relu"
)(hidden)
```

```
# Output 1
delivery_time = Dense(
    1,
    activation="linear",
    name="Delivery_Time"
)(hidden)
```

```
# Output 2
delivery_rating = Dense(
    1,
    activation="linear",
    name="Delivery_Rating"
```

```

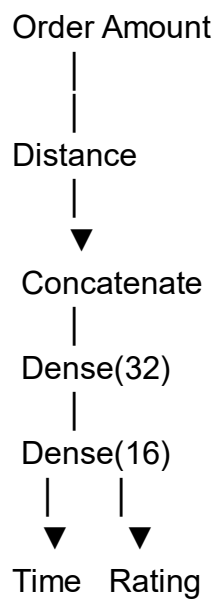
)(hidden)

model = Model(
    inputs=[amount, distance],
    outputs=[delivery_time, delivery_rating]
)

model.summary()

```

Architecture



Task 3: Convert Sequential Model to Functional API

Sequential Version

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(64, activation="relu", input_shape=(20,)),
    Dense(32, activation="relu"),
    Dense(10, activation="softmax")
])

```

Functional API Version

```
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
```

```
inputs = Input(shape=(20,))
```

```
x = Dense(
    64,
    activation="relu"
)(inputs)
```

```
x = Dense(
    32,
    activation="relu"
)(x)
```

```
outputs = Dense(
    10,
    activation="softmax"
)(x)
```

```
model = Model(
    inputs=inputs,
    outputs=outputs
)
```

```
model.summary()
```

Why Functional API?

It is more flexible because it supports:

- Multiple inputs
 - Multiple outputs
 - Skip connections
 - Shared layers
 - Complex architectures (e.g., ResNet, U-Net)
-

Task 4: Skip Connection (IPL Score Predictor)

Inputs

- Runs
- Overs
- Wickets

Output:

- Predicted Score

```
from tensorflow.keras.layers import Input, Dense, Add
from tensorflow.keras.models import Model
```

```
inputs = Input(shape=(3,))
```

```
# Hidden Path
```

```
x = Dense(
    16,
    activation="relu"
)(inputs)
```

```
hidden_output = Dense(
    1,
    activation="linear"
)(x)
```

```
# Direct Path (Skip Connection)
```

```
direct_output = Dense(
    1,
    activation="linear"
)(inputs)
```

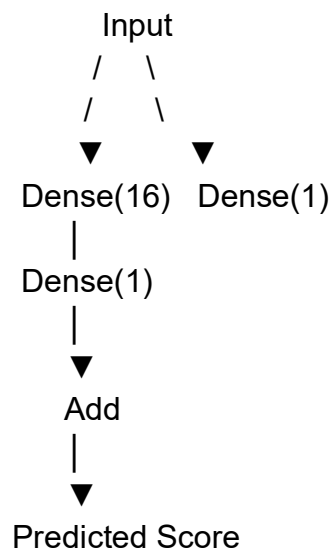
```
# Add Outputs
```

```
final_output = Add()(
    hidden_output,
    direct_output
)
```

```
model = Model(
    inputs=inputs,
    outputs=final_output
)
```

```
model.summary()
```

Skip Connection Diagram



The skip connection allows the input to contribute directly to the final prediction while also passing through hidden layers. This helps preserve information and improves gradient flow in deeper networks.